

School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
BSc. In Computer Science

Project Title:
**Classifying Benign and
DDoS packets**

Supervisor:
Huan Zhang

Student Name:
Ritik Saini

Final Year
Undergraduate Project 2024/25



Date: 17/03/2025

Abstract

This project is aimed to develop a machine learning model for classification of DDoS (distributed denial of service) packets and Benign (Safe) packets in each dataset. Attacks using the DDoS packets are known as the DDoS attacks. These attacks pose a major threat to the network security, as these malicious packets disrupt the network and services (Mirkovic & Reiher, 2004). Detection of these packets have become a necessity to mitigate any future threats. Using publicly available datasets (Sharafaldin, Habibi Lashkari & Ghorbani, 2018) and multiple machine learning techniques, this model was trained to make highly accurate predictions of DDoS packets in order to prevent any disruption on the network for users. The dataset consisted of both malicious and legitimate packets, which was pre-processed (Wang and Lu, 2018), making it well spread. This provided the model with large information to train upon resulting in adaptability of model in various situations and preventing a static model. Many machine learning models, which includes the K-nearestNeighbour model, Decision tree model and the Gaussian Naïve-bayes model were trained and tested with the selected dataset. Each one of the models performed well in the prediction of DDoS packets on the testing dataset. These results were then combined by taking a mean value of all the models to make the final multi layered model improving its efficiency and accuracy to classify the packets. The final model achieved accuracy of 90.18%, proving that machine learning could aid in cybersecurity problems including DDoS packet classification. There were many challenges that had to be addressed in order to make the model flow smoothly, which included handling misbalanced datasets, tuning hyperparameters, data cleaning to remove any object values in dataset and handling null values to ensure that the dataset was uniform. Furthermore, this project 'provides a room for further development and improvisation as a classification system being integrated into a firewall would make it a real-life scanning system. This would prevent any upcoming attacks on the network by dropping any malicious packets found on the server. Moreover, focusing on more concise coding could result in a faster detection software making it suitable for a fast-paced environment. Overcoming the challenges and improvisation of the system could result in a live functioning software which could be very useful for cybersecurity and network's flow. This project portrays the importance of involving machine learning techniques in cybersecurity scene to prevent any unnecessary outage of network due to cyber-attacks. This would result in safer and smoother networks which is desirable in the real world setting and situation to ensure that all the legitimate users are being provided with the security they need and to ensure that the workflow is not hindered.

T able of contents

Chapter 1: Introduction	5
1.1 Context.....	6
1.2 Problem Statement	7
1.3 Aim.....	8
1.4 Motivation.....	8
1.5 Main Objective	8
1.6 Research Questions	9
1.7 Report Structure	9
Chapter 2: Background.....	11
2.1 Traditional DDoS Detection Methods	11
2.2 Machine Learning in DDoS Detection	12
2.3 Feature Selection for DDoS Detection	13
2.4 Challenges of machine learning in DDoS detection	14
2.5 Pros of machine learning in DDoS detection.....	14
Chapter 3: Workflow.....	16
3.1 Requirements.....	16
3.2 Methodology	18
3.3 Implementation	20
3.4 Result.....	25
Chapter 4: Discussion.....	33
4.1 Critical Analysis.....	33
Chapter 5: Further work	35
Chapter 6: Conclusion	36
References	37

T able of Figures

Figure 1:	Columns of Dataset.....	21
Figure 2:	Selected columns.....	22
Figure 3:	Packet distribution.....	22
Figure 4:	Converting object values to integer values	23
Figure 5:	Generating training and testing datasets	23
Figure 6:	Bayes theorem formula	25
Figure 7:	KNN model accuracy	26
Figure 8:	GNB model accuracy	26
Figure 9:	Decision tree model accuracy	27
Figure 10:	Multi layered Random Forest model accuracy.....	27
Figure 11:	Confusion Matrix	28
Figure 12:	Training set packet distribution	29
Figure 13:	Testing set packet distribution.....	29
Figure 14:	Scatter plot of DDoS packets	30
Figure 15:	Scatter plot of Benign packets	31
Figure 16:	Time series analysis.....	31
Figure 17:	Heatmap.....	32

Chapter 1: Introduction

In the current digital era, security is of utmost importance for both organizations and individuals. Distributed Denial of service (DDoS) attacks stand out from other cyber threats as they can make web servers, networks, and online services unusable and unresponsive by flooding them with malicious traffic. These attacks not only impact service availability but also damage business reputations and can lead to significant financial losses (Somani, Gaurav, et al, 2017). Traditional security measures often fail to effectively counter the evolving nature of DDoS attacks as they are highly complex attacks. This arises the necessity of developing a highly adaptive solution.

Machine learning techniques are a proven tool for making systems and software that are of quick learning nature making it a perfect strategy to differentiate between the safe packets (Benign) and malicious packets (DDoS). Machine learning models are known for learning the pattern from a large database. This makes these model work efficiently to detect new types of attacks without an explicit program (Buczak and Guven, 2016). Multiple machine learning models, such as K-nearestNeighbour, Random Forest classifier, Decision tree etc, are known for predicting values after being trained with a well spread dataset. These models recognise pattern in a dataset to make highly accurate predictions. This approach would be beneficial in classifying DDoS packets from Benign packets as a network dataset could provide the models with required pattern. Moreover, being highly adaptive they can learn new patterns of evolving DDoS attacks making the model efficient in real life situations. This could result in a dynamic solution for a high evolving and growing threat to networks.

This project proposes developing a machine learning model for classification of DDoS and Benign packets. A well spread publicly available network dataset consisting of both safe and malicious packets would be used to train the model in order to evaluate and accurately predict the malicious packets in a dataset.

This project is significant as it provides a highly flexible and adaptive solution as compared to the traditional methods, which includes rule-based intrusion detection system where the attack pattern is scanned manually. Machine learning models are exposed to a variety of data, making them a promising avenue for real-time DDoS detection (Buczak and Guven, 2016). This project could possibly contribute for development of an advanced system for attack detection to enhance network's health and security. This could include integrating the machine learning system into a firewall. This would keep detecting the network for any possible attacks in real time which would be a highly working solution and provide organizations to safeguard their network and system from complex cyber-attacks.

1.1 Context

Data in network is transferred in form of a packet. These packets carry important data from source to destination. These safe packets are known as Benign packets. During a cyber-attack, DDoS attack, malicious packets are bombarded into the server making it unusable by legitimate users. This makes the system unavailable and affects organizations and individuals' reputation and finances.

Distributed Denial-of-Service (DDoS) attacks have advanced in recent years, posing significant risks to businesses of all sizes. Typically, these attacks cause difficulties in handling legitimate requests on the target's servers. The main types of DDoS attacks include volumetric attacks, protocol attacks, and application-layer attacks (Singh, Anshuman, and Brij B. Gupta, 2022).

Traditional strategies such as rate limiting, traffic filtering, and IP address blocking are not very successful in countering advanced and large-scale DDoS attacks. Conversely, machine learning (ML) has gained attention for its ability to analyse large volumes of network data and detect complex patterns that may indicate potentially harmful behaviours. Machine learning models have the capability to learn how to recognize abnormalities in network data, resulting in a more pre-emptive and adaptable approach to identifying and stopping DDoS attacks (Doriguzzi-Corin, Roberto, et al, 2020).

The performance of these machine learning models is highly dependent on the datasets that are used to train these models. Commonly, network datasets which comprise of Flow ID, source IP, destination IP, packet count are highly preferred for classification purpose (Buczak and Guven, 2016). These features of machine learning models help detect any possible attack.

A range of learning algorithms, including Random Forest, Neural networks etc are used to detect DDoS attacks varying in success (Santos, Reneilson, et al, 2019). Still, many challenges persist such as misbalanced datasets, which could result in an overfitting or underfitting model. Feature selection is also a challenge which is a crucial part in training a model, as right features to classify packets are necessary to improve the accuracy of model. Moreover, availability of multiple algorithms and models results in difficulty to choose the right model for each solution.

This project focuses on evaluating different machine learning models to classify between DDoS packets and Benign packets by selecting the right features of a dataset to improve accuracy and efficiency with which the model works. Moreover, tuning the hyperparameters to make the model work smoothly in a fast-paced environment.

1.2 Problem Statement

Distributed denial of service (DDoS) attacks being a high evolving cyber-attack poses great threat to network's workflow as the traditional techniques to mitigate the threat often fail to detect the attack patterns. Traditional techniques include manual classification, Intrusion detection system relying on a rule based or signature-based detection methods (Behal and Kumar, 2017), is not an efficient approach as the cybercriminals employ a highly complex attack using hard to detect strategies. Hence there is a high need for a highly adaptable software system which would overpower these strategies making the network safe and immune to these attacks. Machine learning is a promising solution to detect any anomaly in network's traffic. However, many challenges still remain in machine learning detection system. This ranges from a

- High false positive and false negative values, meaning machine learning system wrongly classifying packets in network traffic.
- Misbalanced dataset, which is used for training the model, results in overfitting or underfitting models resulting in models performing correctly on the training dataset but failing to perform well on testing dataset or new attack patterns (Buczak and Guven, 2016).
- Efficiency issues and data preprocessing. While data preprocessing it is required to select features that will be helpful in recognising the pattern of packets. Otherwise, it would affect the accuracy and prediction of models. Alongside, models need to process large amount of data hence using the right algorithms and models are beneficial in making the model work efficiently on the large datasets while in a fast-paced environment (Parashar, Anubha, et al., 2023).

This project will address all these challenges to develop a machine learning system which would be efficient. Multiple models will be used to ensure a highly accurate model and feature selection will be done logically to improve attack detection accuracy. Reducing false positive rates will improve scalability to hence this would optimize the system to be a beneficial in a live working environment to prevent cyberattacks.

1.3 Aim

The aim of this project is to successfully develop and deploy a machine learning based system for classification of network packets and detecting malicious DDoS packets to prevent a DDoS attack. Machine learning techniques will be used to ensure an improvement in attack detection in network as these techniques and models are constantly learning about attack patterns from live networks after being trained using a large dataset. These models would be used to identify real time attacks. The cybercriminals use complex strategies to prevent any detection, yet a machine learning model could prevent such attacks by learning the behaviour and pattern in attacks, using its highly complex algorithms.

1.4 Motivation

Huge increase in cyberattacks, more specifically DDoS attacks have been a critical issue. Being involved in Machine learning and Security engineering modules at university attracted me towards this concern. Reading about the traditional techniques to solve this issue motivated me to base my final year project about this topic. This project is also a way for me to deepen my understanding of machine learning techniques and network functionality to enhance my skills in cybersecurity field. This project is motivated to develop a machine learning system that can:

- Highly improve the accuracy of models for detection of malicious packets by training the model with a balanced dataset and logically preprocessing the data.
- Optimize the model by feature selection such that the complexity of the model reduces while improving the scalability to make it more effective in real-time situations.
- Use results from multiple machine learning models in order to finalize a decision for authenticity of packet instead of using just a single model.

1.5 Objectives

- To develop a machine learning system that would differentiate between Benign and DDoS packets with high accuracy.
- To collect and preprocess network dataset for training and testing the classification model.
- Feature selection to ensure the scalability of model is improved.
- To implement and evaluate multiple machine learning algorithms to effectively classify DDoS packets from Benign packets
- To assess the model's accuracy and precision to evaluate performance

1.6 Research Questions

Q. How accurately does the machine learning model differentiate between a normal network packet and a malicious packet in a network traffic?

- Investigating performance of various machine learning algorithms (Mohammed, Ameer Sameer Hamood, 2024) would answer this question. This project aims to provide a highly accurate model that would predict values efficiently using a balanced training dataset. This model is further tested on an unbalanced testing dataset to check its accuracy with a foreign dataset.

Q. Which feature is considered or ignored while feature selection to improve attack detection?

- Investigating the dataset and logically selecting from the features provided would help in identifying the most relevant traffic attributes that would improve the accuracy and performance of the model. This needs to be done either manually or running some algorithms to find pattern similarities in a pool of data.

Q. What is the significance of balancing dataset on DDoS detection while training the model?

- A balanced dataset is necessary in order to ensure that the model is trained well enough for predicting with high accuracy even on new datasets or a live network. Imbalanced dataset could cause overfitting or underfitting models which could lead to unwanted responses from model. Techniques such as oversampling and under sampling could be beneficial to handle imbalanced datasets.

Q. What is the significance of quality of dataset and preprocessing in affecting the accuracy of a detection model?

- Quality of dataset is crucial as it affects the overall quality of model as it is trained using this dataset hence a poor-quality dataset would result in a model which is underperforming and not suitable for detection in real time. Data cleaning and preprocessing plays an important role while model is being trained as the dataset is supposed to be uniform and any object values would result in unexpected response from the model. Hence data selection and preprocessing majorly affects the model's performance.

1.7 Report Structure

This is a structured report going through every aspect of the project for better understanding of technical aspects and research needed for the project. The report starts off with an 'Abstract' section which basically summarizes the whole project outside of the whole report.

Then the first section introduces the project along with the main aim of the project, problem statement and motivation of the project. Moreover, the objective of the project is also well defined in this section along with the main context and some research questions.

The second section explains in detail about the background of the topic. This consists of multiple section basically summarizing all the research aspect of this project which are needed to be known in order to understand the basic ideology of the project.

The third section is the workflow of the project. This is an important section as this section dives deeper in the technical part of the project explaining in detail the steps taken to achieve the desired model in the 'implementation' section. Moreover, theoretically backed explanations about every step is done in 'methodology' section. Discussion about testing and evaluation is also done in this section along with snippets of resulting graphs and comparison maps. Furthermore, functional and non-functional requirements are mentioned with their severity for the project.

The next section is a 'discussion' section where the model that is created is being discussed and critically analysed.

Then a 'Further work' section talks about the improvisation of the project that could possibly be done in order to fully deploy the system in real time.

Furthermore, a 'conclusion' section finally concludes all the research and technical facts and gives the last verdict of the project.

Lastly all the references used in the report are mentioned in 'references' section for accessing other people's material that aided in this report.

Chapter 2: Background

Distributed Denial of service (DDoS) attacks are one of the most dangerous threats which is aimed at overflowing a network or a service by bombarding the malicious packets into the network traffic (Zaidi, Syed Ali Nawaz, 2025). This results in making the server unusable, affecting the organization's finances and reputation (Buczak and Guven, 2016). This causes a service downtime which disrupts legitimate users' work. DDoS attacks are of three different types, namely Volume based, protocol based, and application layer based.

The volume-based attacks overflow the target network with huge amount of DDoS packets making the server unavailable and causing a huge threat to people's data and workflow.

The protocol attacks usually exploit weaknesses in a protocol of a network. In this the packets are transferred into the network using vulnerabilities of a network.

Application layer attacks generally target an application, which makes the detection of these packets even harder.

Due to an increase in cloud computing, DDoS attacks have become more complex and harder to battle hence a real life highly adaptable solution is required in the cyber scene to prevent any further attacks (Yan, Q. and Yu, F., 2015). There have been traditional methods to detect malicious packets, but they are not efficient taking in account the high paced environment these attacks take place.

2.1 Traditional DDoS attack Detection Methods

There have been many traditional ways to detect any possible malicious packets in the network. Though most of these ways are not efficient enough to scan the network efficiently enough to prevent any future attacks. Hence a highly adaptable solution is required which can be produced by using machine learning models. Some of the traditional methods are:

- Signature Based Detection

This is one of the most traditional detection approaches for DDoS detection. There is a predefined pattern of attacks that is used to understand pattern for new but similar attacks. This approach is highly accurate for already identified attacks as their pattern is already stored in the database, ensuring the same attack could not happen twice. Yet, in the new fast paced world where the pattern of these attacks is constantly changing and the complexity is increasing, this type of approach would not be very beneficial as a highly adaptable solution is more in demand.

- Anomaly Based Detection

This approach focuses on catching any deviation in the normal network. This does have a predefined pattern of data to detect potential malicious packets, rather it focuses on the normal network and catches any packet that is slightly deviated from the established baseline. This could include a sudden change in volume, new packet structures, spike fluctuations etc. This approach is useful to detect and prevent evolving DDoS attacks and is highly adaptable. Yet, its accuracy is not the best resulting in false alarms or classifying many Benign packets as DDoS causing legitimate users and packets to suffer. It needs to be continuously optimized in order to prevent its high misclassification rate.

- Traffic Filtering and rate limiting

These techniques are also traditional way of preventing DDoS attacks by restricting access of certain sources or limiting request threshold.

Rate limiting refers to the technique of limiting requests from a certain address to prevent overflow of packets in server, which highly prevents any volumetric attack and costs low on implementation. But restricting the rate of packets from a certain source would result in blockage of genuine users or data causing the server to be inaccessible during legitimate workflow

Traffic filtering refers to the technique of blocking certain sources by blacklisting them to not provide with any packets. This also blocks any foreign request pattern which is unrecognisable by the server. This is also a low-cost alternative to prevent cyberattacks, yet it could result in blocking genuine sources causing the server to suffer.

Commonly, they are implemented by using a firewall, IP blacklists and web application firewalls (Buczak and Guven, 2016).

Due to these challenges in traditional methods of attack detection, Machine learning poses to be an ultimate solution being highly accurate, efficient and constantly adapting to new situations.

2.2 Machine Learning in DDoS attack detection

Machine learning is a powerful approach to combat cyber-attacks, specifically predicting an upcoming attack. Traditional approaches for detection often fall behind in evolution field, hence requiring constant upgradation to keep up with the new patterns and complex attacks. Whereas machine learning provides a self-evolving solution that can update its database to learn new pattern from past data and could aid in both known and unknown attacks.

Machine learning based system analyses traffic, perform feature selection and classify the packets accordingly using the acquired pattern (Behal and Kumar, 2017), maintaining a high level of accuracy throughout. These models process a

large dataset or network traffic and adapt to any new patterns occurring during attack which gives them an upper hand over the old traditional methods.

These machine learning techniques can be classified into 3 main approaches, namely supervised learning approach, unsupervised learning approach and a hybrid learning approach.

Both supervised and unsupervised approaches require training the model using a certain network dataset to predict future values.

The main difference between the two is that in a supervised learning approach, the dataset used for training the required model consist of labels meaning that both safe packets and malicious packets are provided to the model in order to train efficiently to detect any anomalies in the network. An example of this would be 'random forest', which classifies packets by feature selection. Whereas, for training an unsupervised model, the datasets aren't required to have labels. In place of labels, the model learns the pattern of attack making it highly useful and efficient in detecting any attack with unrecognised patterns. An example of this would be 'autoencoder', which detects any deviation in a normal traffic and classify the packets causing deviation as safe or potential threat.

A hybrid machine learning model basically joins both supervised and unsupervised learning approaches to improve the detection efficiency and accuracy while reducing any misclassification of packets. An example of this type of approach is using a 'random forest' and 'autoencoder' for detecting both known and unknown cyber-attacks as this provides high adaptability of model.

2.3 Feature Selection for DDoS attack detection

One of the most crucial steps while building and training a machine learning based system is feature selection, which ensures high efficiency of the model. This is a critical stage which could determine model's performance and efficiency.

Some of the key features in a network dataset includes packet features and traffic features.

Packet features include data about the packet. This includes arrival time, packet size and distribution, duration of flow etc. A model recognises these features to examine any packet anomalies in a network to prevent malicious packets to reach its target.

Traffic features include data about the network traffic. This includes traffic size, connections, source and destination etc. A model would recognise attack patterns in these features resulting in detection of a possible volumetric attack.

Selecting features which are relevant to detection of attack results in a model that could benefit the server security. However, selecting the relevant features is a huge challenge due to constant evolving nature of both server and cyberattacks. Hence, feature selection must be done logically.

2.4 Challenges of machine learning in DDoS detection

Building a machine learning model might be the ultimate solution for DDoS attack detection, yet there are several challenges that need to be overcome in order to ensure the model works as expected and aid in server security. These challenges range from misbalanced datasets to misclassification of packets affecting model's overall performance.

The most common challenges faced while building these models are:

- Misclassification of packets. This means the model wrongly classifies packets as safe or malicious causing a major problem. This could result in blocking services from legitimate users or letting malicious packets go through without detection. This can be avoided by feature selection as choosing the relevant feature to classify packets could highly prevent this unexpected behaviour of model.
- Feature selection during training models. Feature selection could prevent other challenges while making the model, but feature selection is a big task itself. Logically determining the features that would differentiate between the Benign and DDoS packets is crucial to ensure required model's performance.
- Imbalanced dataset used for training the model. This would cause the difficulty in training the model for predictions with high accuracy. This causes a poor performing model which causes high misclassifications and are not suitable for usage. Moreover, this could cause overfitted or underfitted models which could lead to model performing well on training dataset but not being adaptable and not performing well on testing dataset or real-life networks. DDoS attacks are constantly evolving needing a solution with continuous adaptation which would be hard to achieve if the training is not performed with well spread balanced dataset.
- Lack of high-quality dataset available publicly. Hence dataset selection would have to be cautious to prevent any of these issues to occur while a model is being developed.

Hence, while a machine learning model would improve detection, these challenges need to be addressed in order to maintain high model performance and efficiency. Overcoming these challenges could result in a scalable and highly adaptable solution to one of the biggest cyber threats.

2.5 Pros of machine learning in DDoS detection

After overcoming all the above-mentioned challenges, a model which would be highly beneficial to safeguard a network could be achieved with a lot of advantages over traditional approaches to DDoS attack detection.

These advantages include:

- Better detection rate and low misclassifications. As a machine learning model can analyse large network traffic, attack patterns can be more generalised for identification with high accuracy as compared to traditional techniques. After the model training is done on a well-balanced dataset, misclassification could be reduced while increasing the performance to ensure a better scalable alternative to traditional methods.
- High adaptability of the model. This is a result of model continuously updating their database to recognise new patterns to prevent any evolving threat. This makes the model self-learning and not needing a constant manual intervention.
- fast response time and faster detection. Traditionally, packet classification required manual help which increased the response time making it hard to prevent attacks. The machine learning models consist of complex algorithms making the recognition faster as a certain pattern recognition is done to classify packets. This results in fast responses to prevent attacks.
- Performing well on high traffic volumes and mitigating threats. Being highly evolving in nature, machine learning model would be an ideal solution to mitigate threats on a large network environment, such as cloud computing. Moreover, advancement in these models could help enable an automatic filtering mechanism for rate limiting and blacklisting to prevent any further possible cyberattacks.
- Highly reduces manual work and downtimes. Machine learning models would not require a constant need for manual work to ensure its workflow hence resulting in an automatic security system. This would lead to lesser downtimes benefitting organisations in terms of finances as the network wouldn't be out of service or would be back online way faster than if using traditional approaches.

Keeping all these advantages in mind, it is sure that a working real time machine learning model, after overcoming its challenges, provides great benefit to organisations and networks. Hence, machine learning is being researched more and more to implement in fields like cyber security to make the servers stronger in terms of security as high accuracy, scalability and adaptability. These aspects are very crucial when it comes to combatting a cybercrime involving complex strategies and constantly evolving methods.

Chapter 3: Workflow

The workflow of this project comprises of several procedures varying from data collection to data cleaning to preprocessing to visualizations. All these steps are explained in detail in this section providing a technical and theoretical overview about the model which is developed in this project.

A workflow is beneficial to be traced out before starting building a model as it provides a basic blueprint or outline of how to implement machine learning practices and achieve the desired product. This makes it an important step to approach a modern problem.

3.1 Requirements

Before any of the technical or theoretical aspect, requirement of the project needs to be determined in order to have a clear view of how the product should behave. These requirements must be met in order to produce a model which will combat ant problem we are trying to solve by overcoming any challenges and making most of the advantages it has. There are 2 types of requirements namely functional and nonfunctional requirements.

3.1.1 Functional requirements

Functional requirements explain the task of model that are supposed to be done accurately in order to make the model behave in the desired way. These requirements lay out a basic structure of functionalities that the model should or must perform

Following are some Functional requirements for a model to classify between Benign and DDoS packets in a network:

Number	Requirements	Must have / should have / Could have / Won't have
1	Data Collection and Preprocessing	Must have
2	Machine Learning Model Development	Must have

3	Real-time Traffic Classification	Must have
4	Firewall Integration	Could have
5	Dynamic Rule Update	Should have
6	Performance Metrics	Could have
7	Model Retraining and Updates	Could have
8	Packet classification	Must have
9	Feature selection	Must have
10	Alerts	Could have

3.1.2 Non-Functional Requirements

Non-Functional Requirements lays out a basic structure for performance of a system. These requirements focus more on qualitative attributes of a model.

Following are some non-functional requirements:

Number	Requirements	Must have / should have / Could have / Won't have
1	Scalability	Must have
2	Low Latency	Must have
3	Performance Optimization	Should have
4	Privacy Compliance	Must have

5	Usability and Interface	Could have
6	Logging and Monitoring	Should have
7	Compatibility	Should have
8	Reliability	Must have

3.2 Methodology

Understanding the basic methodologies is required in order to build a model to produce a solution. These methodologies include basic working of a model and crucial steps that are needed to be carried out in order to build a highly compatible model. The basic understanding of steps and knowledge required poses a smooth flow of work during building a model.

Methodologies required to understand this model are as follows:

3.2.1 Necessary libraries

This model is built in Python language. Python offers a variety of modules or libraries which consist of algorithms, functions or set of codes that aids in building a highly complex and code dense projects.

There are certain libraries that mainly focuses on the machine learning aspects, like complex data analysis, machine learning techniques and visualization of data in order to reduce the complexity of code needed.

Several libraries such as 'Pandas', 'Matplotlib', 'Sci-kit learn', 'Numpy' and 'Seaborn' are used in this model to achieve the desired model. These are widely used libraries for machine learning purposes as they help reading the dataset, converting to data frames, feature selection, model training, model building, visualization, testing and evaluation conveniently.

Using these libraries prevent from writing complex codes and algorithms and using predefined functions makes code easy to understand and read.

3.2.2 Data selection

For building a model to classify between DDoS packets and Benign packets, dataset selection is done by selection a network dataset to train the model to learn all the aspects of network and packets. Choosing the right dataset is an important step for model training as this is dataset would teach the model about the patterns to look for.

Also, selecting a dataset that is well balanced is also important to make sure that the model does not overfit or underfit. This could result in the model performing well on the training dataset but suffering to predict accurately on the testing dataset or in real life network. Hence this challenge is a must to overcome before model training. A well balanced and well spread dataset would result in a highly adaptable model which would be useful in a live network environment.

3.2.3 Data preprocessing and feature selection

Preprocessing of data is the next step done to make sure that the data contained in a dataset is uniform. Data cleaning is done in this step by handling null values or duplicates to remove any unwanted or miscellaneous data values. Then any object values are handled to convert to integer values for comparison or to make them readable by the model. Data sets are also balanced, if imbalanced, to make sure the model gets a high-quality dataset while it is being trained.

Feature selection is the next step in machine learning where specific feature is selected from the dataset and only the relevant values in dataset are used to make sure that model does not have to deal with too much irrelevant information.

This improves performance of the model by improving training speed to enhance interpretability. There are algorithms that help select relevant features making this step easier. Model's performance is highly dependant on these steps as they lay the foundation of any machine learning based model.

3.2.4 Choosing and training the model

Choosing the right model is essential to evaluate the train data to make predictions with high accuracy on test data. This is a crucial part to solve a given problem/predict values with high accuracy as the model is chosen as per the issue which needs to be resolved. There are multiple machine learning models having different applications.

Training is done to fit the models using the training dataset to teach the pattern for making further predictions on the test dataset with high accuracy. This is the part where model learns about the training dataset to generate a generalized pattern. While training, the training dataset is split into 2 parts. One of these parts train the model and the other part test the model. After this training the model is introduced to the testing dataset where the model's performance is determined as the testing dataset is different from training dataset making sure that the model works as expected under different conditions.

3.2.5 Testing model

Testing of model is done to check the performance and accuracy of model to see how efficient the model is and if the model is working properly. The testing of model is usually depicted using graphs or other visual methods. This is where the model's performance is captured and represented.

If model's performance is not up to the mark, the hyperparameters are tuned or other steps are done again to improve model's response. The model is sometimes introduced to several testing datasets to detect any issue in the model which needs to be addressed. Hence testing is the last step in machine learning to ensure a highly accurate model.

3.3 Implementation

This is a technical project which consists of various implementation steps. There is a specific procedure that is followed in order to achieve a machine learning based system. These procedures are explained in detail that how they are performed during the project. These steps are as follows:

3.3.1 Importing dataset and libraries

Python is the main language used for machine learning purposes hence importing certain python-based libraries a necessary. There are plenty of libraries that are used to build the DDoS detection model.

For reading the dataset and converting the required dataset to a data frame, 'Pandas' library is used. This is a crucial library widely used for data analysis. It provides with functions to read a dataset or a csv file and provides a data frame that is further used for training and testing models.

'Sci-kit learn' library is used which is a crucial library used for machine learning and further data analysis. Most of the models, encoders, classifiers etc are imported from within this library for this project. This library consists of various functions which are helpful for data mining, data processing and model building. Data preprocessing, data cleaning and training the models are done in this project using this library hence proven to be one of the most important machine learning libraries.

For numerical computations, 'Numpy' library was used. This library consists of functions that helps in numerical computations using complex algorithms for analyzation and manipulation of datasets. The mathematical operations are done by using numpy library in this project. This is important in machine learning as data manipulation is an important aspect for model building.

For visualisation of the results, 'Seaborn' and 'Matplotlib' are used. These libraries help generate visual graphs that helps to easily understand the model performance. Seaborn library is used to produce the heatmaps for correlation of various columns. Matplotlib library was used to plot various graphs and maps with customizations to represent the results as needed. This is essential in comparing multiple datasets, values, relation between variables and plot them graphically. This basically includes bar graphs, pie charts, scatter plots etc. Many graphs are presented in this report in result section where the performance of model is visually plotted.

After importing libraries, dataset was imported, and a data frame was made using Pandas library. 2 publicly available network datasets are being used in this project. One for training the model and other was testing. Data frame for both was created for further analysis and model building. The dataset was carefully chosen to prevent underfitting or overfitting. Training dataset is balanced to ensure that the model is trained well, and testing dataset is imbalanced to check if model can work under different conditions.

3.3.2 Data preprocessing and feature selection

Data preprocessing and feature selection is done by selecting only the required columns from the datasets which will be used to train the machine learning models. The datasets selected are from <https://www.kaggle.com/datasets/devendra416/ddos-datasets>.

The balanced training dataset has around 12.7 million rows, out of which only 700,000 were selected. Similarly, the unbalanced testing dataset has around 7.6 million rows, out of which again only 700,000 were selected. Both datasets have 85 columns (features).

The dataset consisted of the following columns

```
# retrieving column names of dataset
dataset.columns

71
.. Index(['Unnamed: 0', 'Flow ID', 'Src IP', 'Src Port', 'Dst IP', 'Dst Port',
'Protocol', 'Timestamp', 'Flow Duration', 'Tot Fwd Pkts',
'Tot Bwd Pkts', 'TotLen Fwd Pkts', 'TotLen Bwd Pkts', 'Fwd Pkt Len Max',
'Fwd Pkt Len Min', 'Fwd Pkt Len Mean', 'Fwd Pkt Len Std',
'Bwd Pkt Len Max', 'Bwd Pkt Len Min', 'Bwd Pkt Len Mean',
'Bwd Pkt Len Std', 'Flow Byts/s', 'Flow Pkts/s', 'Flow IAT Mean',
'Flow IAT Std', 'Flow IAT Max', 'Flow IAT Min', 'Fwd IAT Tot',
'Fwd IAT Mean', 'Fwd IAT Std', 'Fwd IAT Max', 'Fwd IAT Min',
'Bwd IAT Tot', 'Bwd IAT Mean', 'Bwd IAT Std', 'Bwd IAT Max',
'Bwd IAT Min', 'Fwd PSH Flags', 'Bwd PSH Flags', 'Fwd URG Flags',
'Bwd URG Flags', 'Fwd Header Len', 'Bwd Header Len', 'Fwd Pkts/s',
'Bwd Pkts/s', 'Pkt Len Min', 'Pkt Len Max', 'Pkt Len Mean',
'Pkt Len Std', 'Pkt Len Var', 'FIN Flag Cnt', 'SYN Flag Cnt',
'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt', 'URG Flag Cnt',
'CWE Flag Count', 'ECE Flag Cnt', 'Down/Up Ratio', 'Pkt Size Avg',
'Fwd Seg Size Avg', 'Bwd Seg Size Avg', 'Fwd Byts/b Avg',
'Fwd Pkts/b Avg', 'Fwd Blk Rate Avg', 'Bwd Byts/b Avg',
'Bwd Pkts/b Avg', 'Bwd Blk Rate Avg', 'Subflow Fwd Pkts',
'Subflow Fwd Byts', 'Subflow Bwd Pkts', 'Subflow Bwd Byts',
'Init Fwd Win Byts', 'Init Bwd Win Byts', 'Fwd Act Data Pkts',
'Fwd Seg Size Min', 'Active Mean', 'Active Std', 'Active Max',
'Active Min', 'Idle Mean', 'Idle Std', 'Idle Max', 'Idle Min', 'Label'],
dtype='object')
```

Figure 1 : columns of dataset

Out of these columns only the followings were selected

```
# Selecting the necessary columns needed for data pre processing and retrieving information

attrs = ['Flow ID','Timestamp','Fwd Seg Size Min','Src IP','Dst IP','Flow IAT Min','Src Port','Tot Fwd Pkts','Init Bwd Win Byts','Label']
dataset = dataset[attrs]
testset = testset[attrs]
dataset.info()
```

```
[8]
...
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700000 entries, 0 to 699999
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Flow ID                700000 non-null object
1   Timestamp              700000 non-null object
2   Fwd Seg Size Min       700000 non-null int64
3   Src IP                 700000 non-null object
4   Dst IP                 700000 non-null object
5   Flow IAT Min           700000 non-null float64
6   Src Port               700000 non-null int64
7   Tot Fwd Pkts           700000 non-null int64
8   Init Bwd Win Byts      700000 non-null int64
9   Label                  700000 non-null object
dtypes: float64(1), int64(4), object(5)
memory usage: 53.4+ MB
```

Figure 2 : selected columns

Out of these features, only some are relevant for model building. These columns were selected carefully to maximise model's adaptability while reducing the amount of data that is presented while training. The basic information presented in the dataset were basically about the network packets including timestamp, Flow ID, Source and Destination IP etc.

Main columns which are selected are Source IP, Destination IP, Timestamp, Label, Flow ID etc. These columns are relevant for recognising any pattern emerging during the attacks or unexpected behaviour by network.

There were 2 datasets, for training and testing. Training set was balanced, and testing set was imbalanced. The following snippet shows the distribution of packets in both the sets.

```
# performing a value count to check the number of ddos and benign packets in training dataset

dataset['Label'].value_counts()
```

```
[10]
...
Label
ddos      372648
Benign    327352
Name: count, dtype: int64
```

```
testset['Label'].value_counts()
```

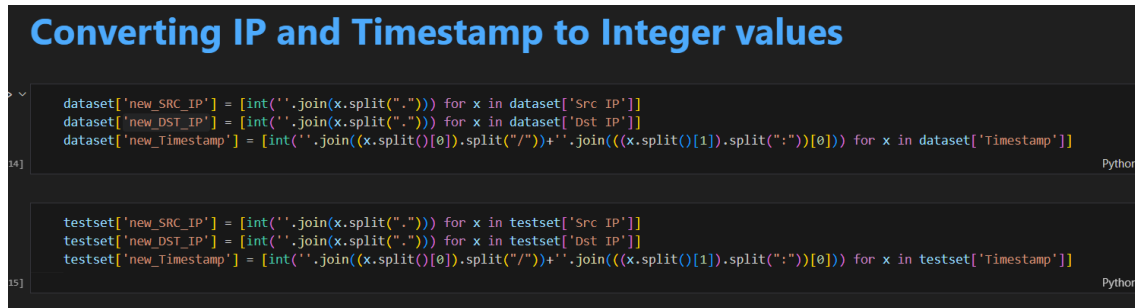
```
[11]
...
Label
Benign    605470
ddos      94530
Name: count, dtype: int64
```

Figure 3 : Packet distribution

After feature selection, data cleaning is done to ensure that the dataset provided does not include any null values or object values.

Source IP, Destination IP and Timestamp column values are converted into integer values from object formats for comparisons and pattern recognition. This was done by removing all the period (.), colons (:), slashes (/) and AM/PM. The numbers were then combined to make a single integer value. These new values were stored in new columns of source IP, destination IP and timestamps while the original columns were removed to make the dataset concise.

The following code was used to convert



Converting IP and Timestamp to Integer values

```
dataset['new_SRC_IP'] = [int(''.join(x.split("."))) for x in dataset['Src IP']]
dataset['new_DST_IP'] = [int(''.join(x.split("."))) for x in dataset['Dst IP']]
dataset['new_Timestamp'] = [int(''.join((x.split()[0]).split("/")+'''.join((x.split()[1]).split(":"))[0])) for x in dataset['Timestamp']]

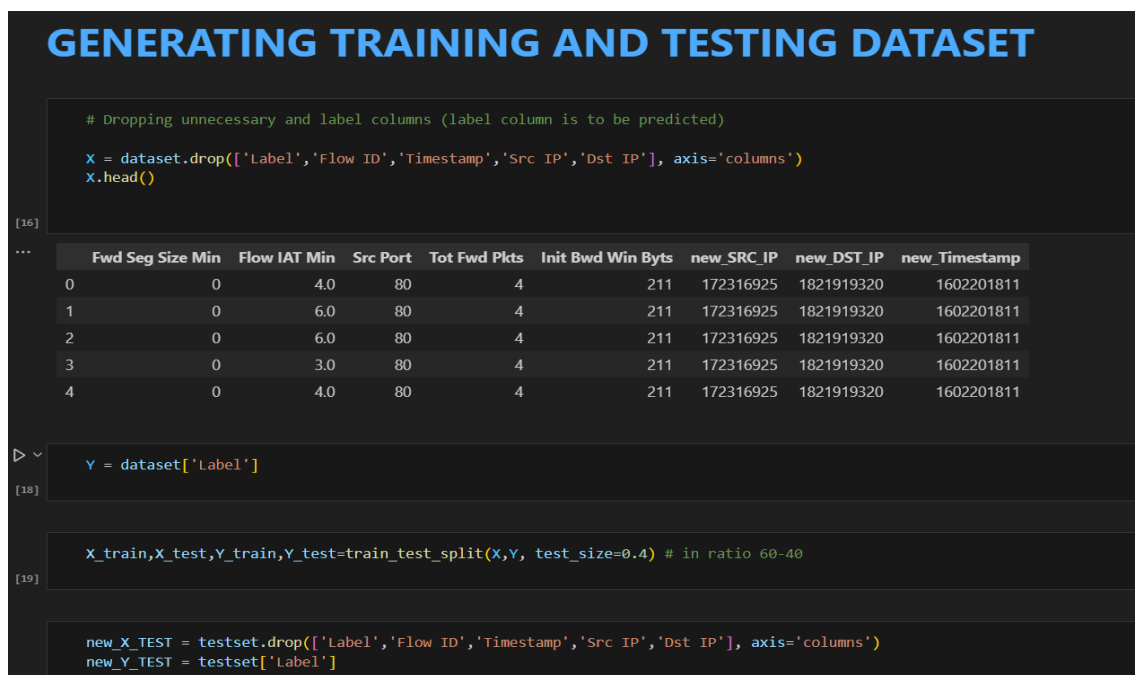
testset['new_SRC_IP'] = [int(''.join(x.split("."))) for x in testset['Src IP']]
testset['new_DST_IP'] = [int(''.join(x.split("."))) for x in testset['Dst IP']]
testset['new_Timestamp'] = [int(''.join((x.split()[0]).split("/")+'''.join((x.split()[1]).split(":"))[0])) for x in testset['Timestamp']]
```

Figure 4 : Converting object values to integer values

3.3.3 Generating testing and training datasets

The training dataset was divided into 2 datasets after removal of unnecessary columns, one contained the labels i.e. authenticity of packets and other contained packet information except labels. This was further split into train and test subsets by train-test split. This was in ratio of 60-40 meaning the whole training dataset was divided into 60% training and 40% testing. These subsets will be used to train and test the model using the training dataset.

The testing dataset will be used later to check for model's adaptability in new and unknown environments or networks.



GENERATING TRAINING AND TESTING DATASET

```
# Dropping unnecessary and label columns (label column is to be predicted)
X = dataset.drop(['Label', 'Flow ID', 'Timestamp', 'Src IP', 'Dst IP'], axis='columns')
X.head()
```

	Fwd Seg Size Min	Flow IAT Min	Src Port	Tot Fwd Pkts	Init Bwd Win Byts	new_SRC_IP	new_DST_IP	new_Timestamp
0	0	4.0	80	4	211	172316925	1821919320	1602201811
1	0	6.0	80	4	211	172316925	1821919320	1602201811
2	0	6.0	80	4	211	172316925	1821919320	1602201811
3	0	3.0	80	4	211	172316925	1821919320	1602201811
4	0	4.0	80	4	211	172316925	1821919320	1602201811

```
Y = dataset['Label']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.4) # in ratio 60-40

new_X_TEST = testset.drop(['Label', 'Flow ID', 'Timestamp', 'Src IP', 'Dst IP'], axis='columns')
new_Y_TEST = testset['Label']
```

Figure 5 : Generating training and testing datasets

3.3.4 Building Model

There are plenty of models available that are used to predict values on the testing dataset. For this project, multiple models are used and then the result is calculated by a classifier. These models are highly complex machine learning models which are imported from libraries. Models that are used for building the final multi layered model are K-nearestNeighbour model, Decision tree model and Gaussian naïve bayes model.

- ❖ K-nearestNeighbour model used Euclidean distance to predict values. It uses the formula to find the distance between the number of neighbours given and the new data point. The nearest neighbour is then calculated, and the new data point is classified accordingly. Its performance is dependent on the k value provided. This model was chosen as it uses a highly effective learning algorithm. This model was chosen because of the following key advantages:
 - Works well in detecting outliers due to efficient algorithm. This makes the model adaptive when there is a change in traffic since it can dynamically classify new attack types.
 - Highly accurate for a well spread data, making the model effective in distinguishing between attack and normal traffic
- ❖ Decision tree is a hierarchical model, which makes a tree structure having nodes, branches and leaves using feature values. Different attributes are branched out with the maximum gain. The nodes represent feature values, branches represent outcome, and leaves represent the result. It is a supervised learning algorithm basically used for regression and classification tasks. This is a recursive approach where the best feature is used for splitting the data until it reaches the result. This model was chosen because of the following key advantages:
 - Doesn't require feature scaling, hence works well with imbalanced or not standardized data. This is beneficial in detecting multiple types of DDoS attacks.
 - Has a built-in feature selection hence this step is done automatically. This helps detecting any abnormal spikes in the network preventing any future DDoS attack.
- ❖ A Gaussian naïve bayes is a model based on bayes theorem. It works on principle of probability. The highest probability is selected as the result for a class value. In this model, it is assumed that the feature values are of normal distribution and are independent. The bayes theorem is used to calculate the probability.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Figure 6 : Bayes theorem formula

Here, A is the class and B is the feature. This model was chosen because of the following key advantages:

- Effective for real-world data as it is efficient in handling continuous data. This helps detecting sudden DDoS attacks by scanning for unusual pattern in network. Moreover, it is a computationally inexpensive model, hence making it is well suited for real time detections.
- Works well with datasets that have multiple features since it is assumed that the features are independent.

The testing and training dataset is provided to these 3 different models and predictions are carried out. The majority decision is considered for each packet and is given to the last model, which is the final model. The final model is a Random Forest model. This model was chosen to be the final model as it is an ensemble learning algorithm which uses multiple decision trees hence making it a lot more accurate. It is basically used for classifications, like in this model. This is the multi-layered model which uses above results to predict values. The above values are compared and the majority result for a packet is considered. The majority result is then used to train the final model which is then tested on the testing set.

3.4 Result

Testing is done to check if the models are working and how accurately is the data predicted by each model and by the final model. Visual representation of dataset is also done in order to easily compare the values within dataset and to check for other patterns and correlations between various columns.

3.4.1 Accuracy

Accuracy of a model refers to how accurately the model can predict values. Accuracy ensures that the model is working up to the mark and can detect if a model requires more tuning

The accuracy of final model is 90%, precisely 90.18. The accuracy of other layered models was also 90%. Being highly accurate, this model can correctly classify between Benign and DDoS packets in a network. This was achieved as the datasets used and feature selection was done logically resulting in a highly accurate model.

Accuracy of each model:

Knearestneighbor model -

KNN Classifier

```
[21] knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, Y_train)

... KNeighborsClassifier
KNeighborsClassifier()

[22] knn_accuracy = accuracy_score(new_Y_TEST, knn.predict(new_X_TEST))
print(f"KNN Accuracy: {knn_accuracy*100:.2f}%")

... KNN Accuracy: 90.18%
```

Figure 7 : KNN model accuracy

Gaussian Naïve-Bayes Model -

Gaussian Naive-Bayes

```
[23] gnb = GaussianNB()

gnb.fit(X_train, Y_train)

... GaussianNB
GaussianNB()

[24] gnb_accuracy = accuracy_score(new_Y_TEST, gnb.predict(new_X_TEST))
print(f"Naive-Bayes Accuracy: {gnb_accuracy*100:.2f}%")

... Naive-Bayes Accuracy: 90.18%
```

Figure 8 : GNB model accuracy

Decision tree model -

Decision Tree

```
[25] dt = DecisionTreeClassifier()

dt.fit(X_train,Y_train)

... ▾ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()

[26] dt_accuracy = accuracy_score(new_Y_TEST, dt.predict(new_X_TEST))
print(f"Decision Tree Accuracy: {dt_accuracy*100:.2f}%")

... Decision Tree Accuracy: 90.18%
```

Figure 9 : Decision tree model accuracy

Multi layered Random Forest model -

```
[32] rf = RandomForestClassifier(n_estimators=200)
rf.fit(final_X_input,final_y_input.values.ravel())

... ▾ RandomForestClassifier ⓘ ?
RandomForestClassifier(n_estimators=200)

[33] rf_accuracy = accuracy_score(new_Y_TEST, rf.predict(new_X_TEST))
print(f"Random Forest Accuracy: {rf_accuracy*100:.2f}%")

prediction = rf.predict(X_train)

... Random Forest Accuracy: 90.18%
```

Figure 10: Multi layered Random Forest model accuracy

Hence, it is observed that

Model	Accuracy
1) Gaussian Naïve-Bayes	90%
2) KNN	90%
3) Decision tree	90%
4) Random Forest	90%

3.4.2 Visual representation

Visually representing a model's performance is helpful in getting a deeper understanding of model's efficiency and accuracy.

- A confusion matrix was plotted to detect the false negative and false positive values for determining how much of the values are being wrongly classified and their ratio.

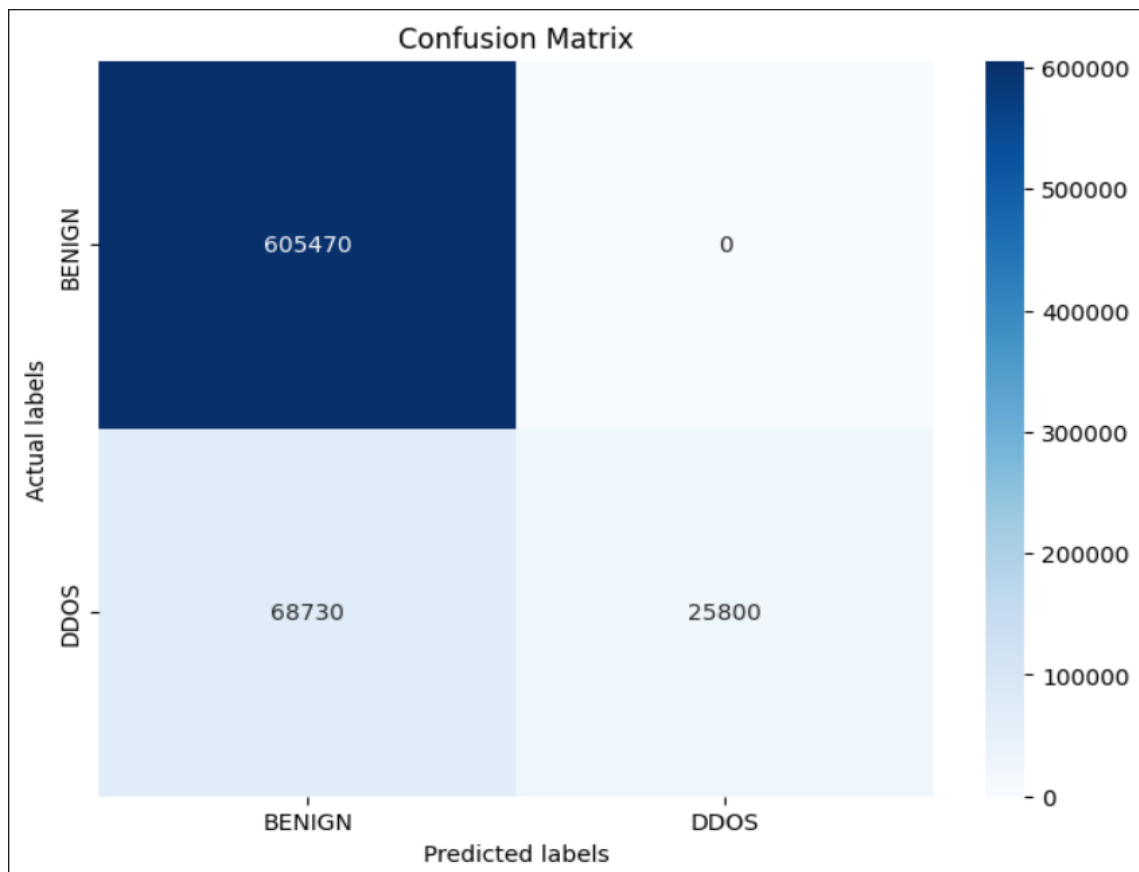


Figure 11 : Confusion Matrix

- The distribution of packets in selected datasets is visually represented by the following pie charts.

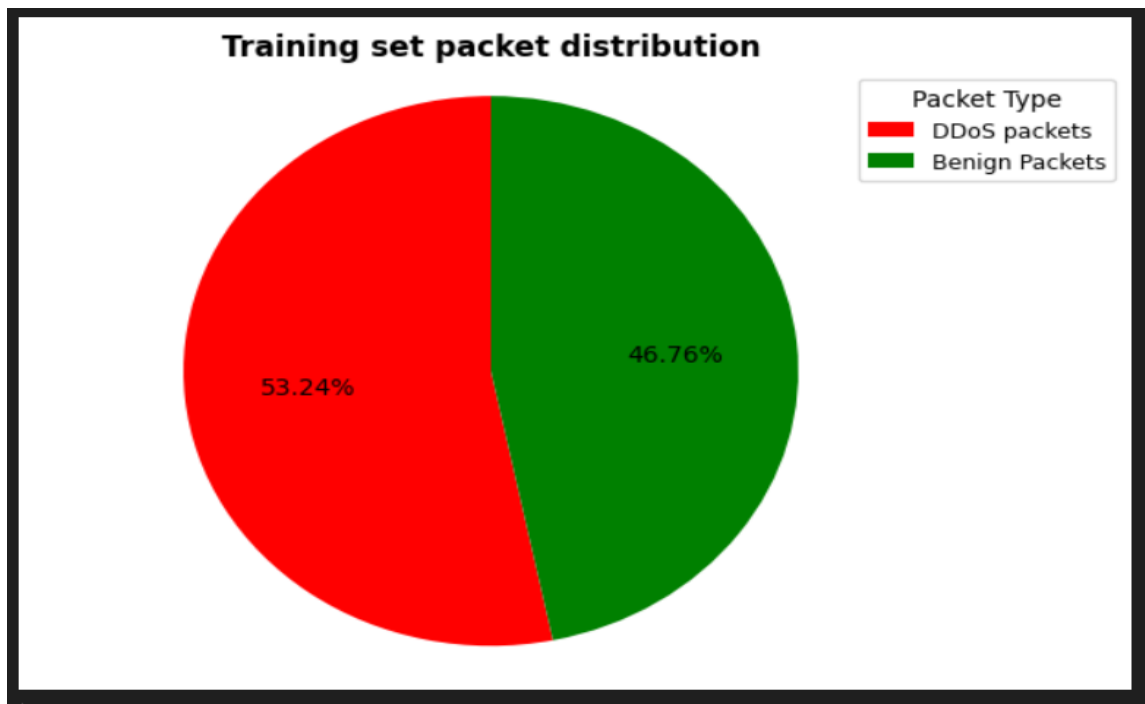


Figure 12 : Training set packet distribution

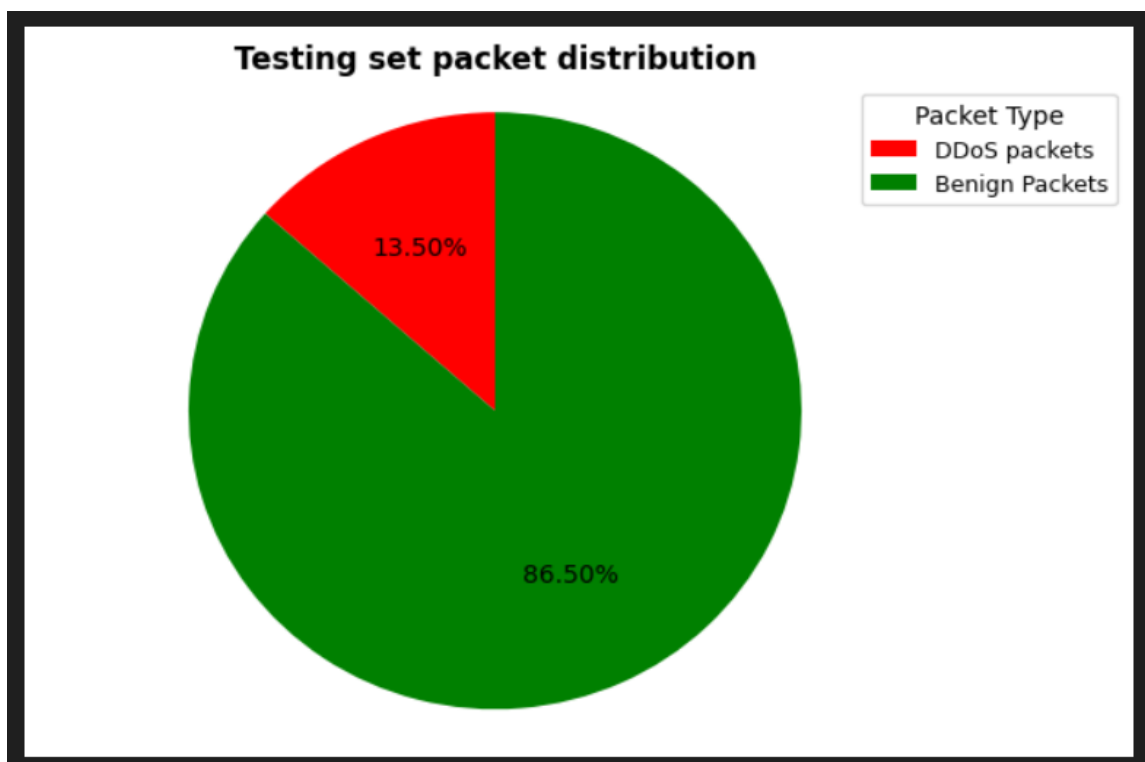


Figure 13: Testing set packet distribution

These above pie charts are highly efficient in understanding that the percentage of packets in balanced training dataset is similar. Whereas in the unbalanced

testing dataset, Majority of dataset packets are Benign as compared to DDoS packets.

This shows that the training dataset was well balanced hence the model produced in adaptable and scored a high accuracy even on an unbalanced testing dataset.

- A scatterplot graph was plotted to understand the dependency of DDoS packets and Benign packets on time, and how well spread they are overtime in a network

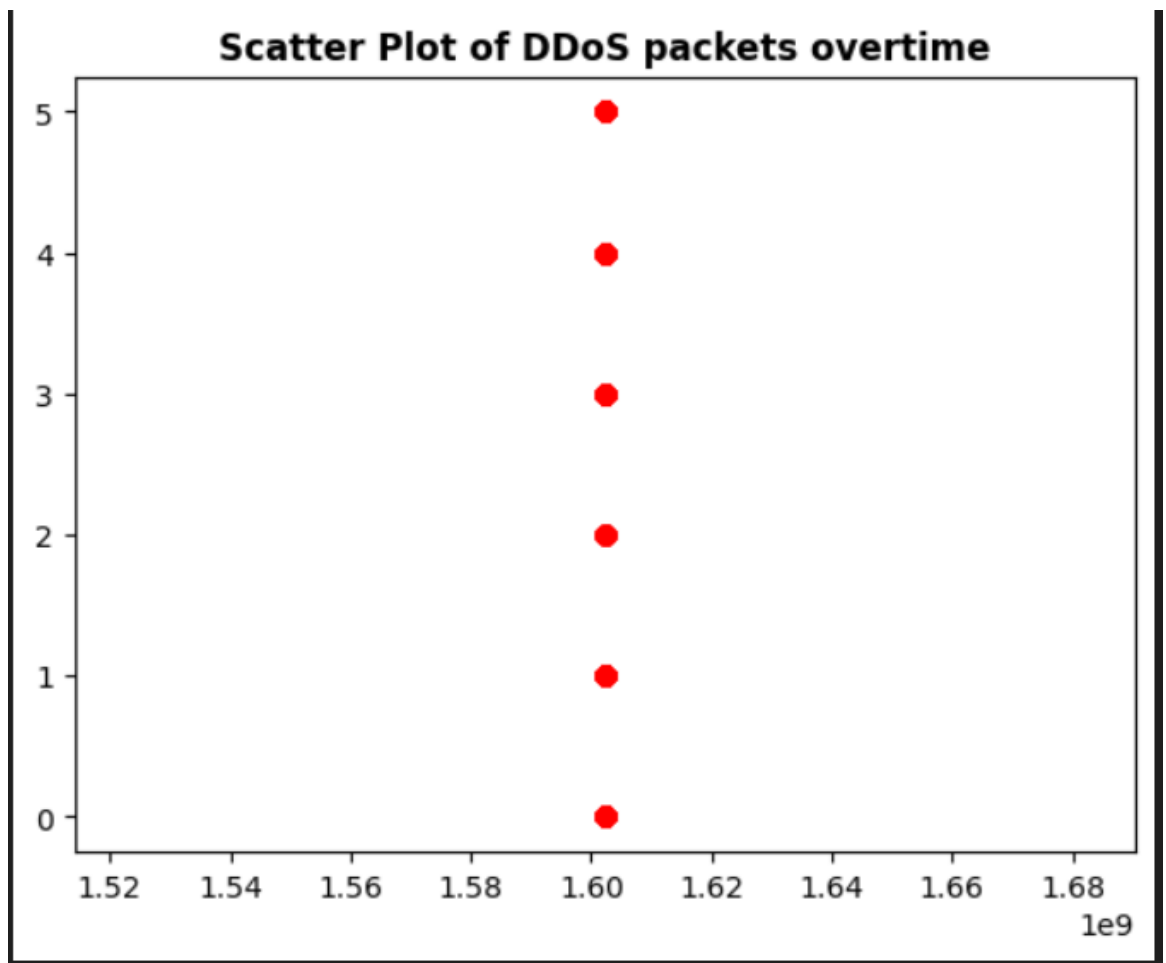


Figure 14: Scatter plot of DDoS packets

This graph shows that the DDoS packets are sent continuously over a specific period, meaning there is a certain pattern in terms of time which is followed by the DDoS packets. This pattern is recognised by the model to prevent any DDOS packets to go undetected.

Whereas in the following graph, it can be observed that the Benign packets are well spread overtime meaning there is no fixed time for these packets.

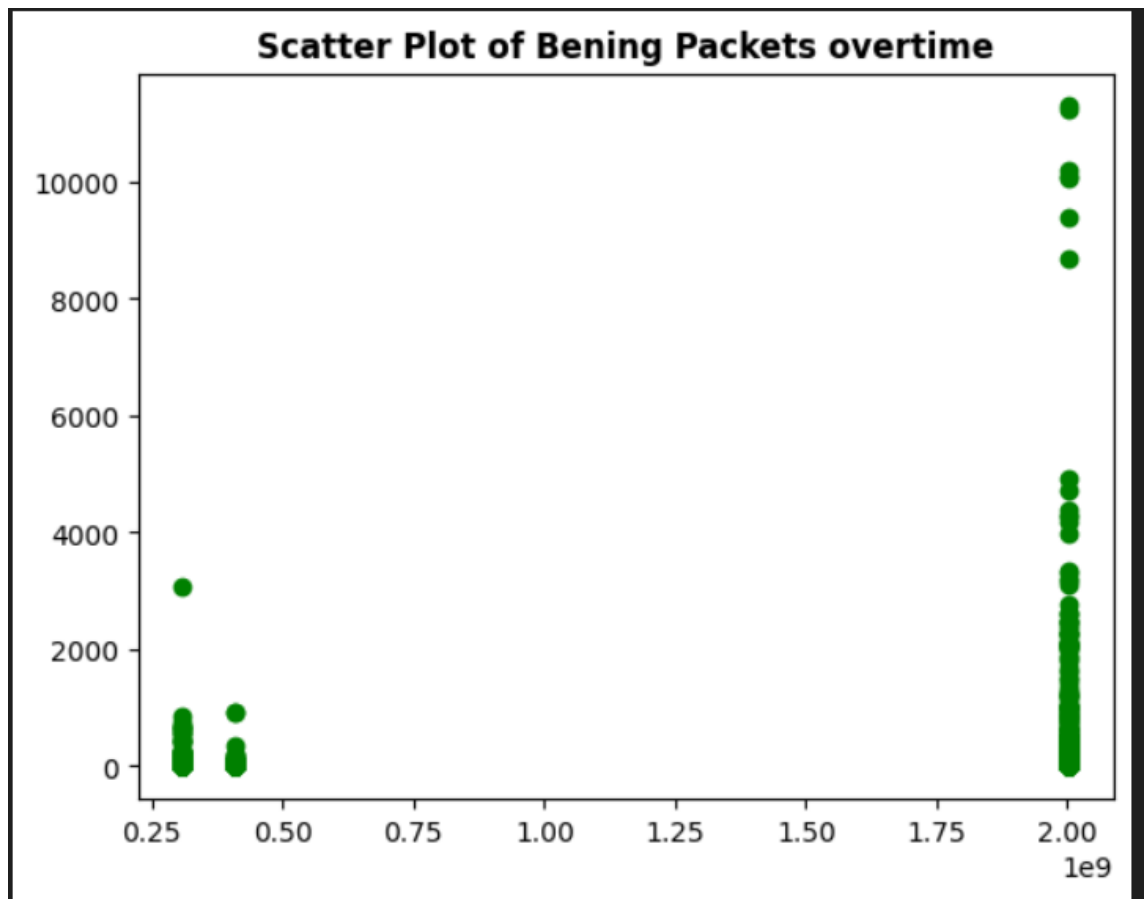


Figure 15 : Scatter plot of Benign packets

- A time series analysis graph was plotted to analyse mean forward packets over time.

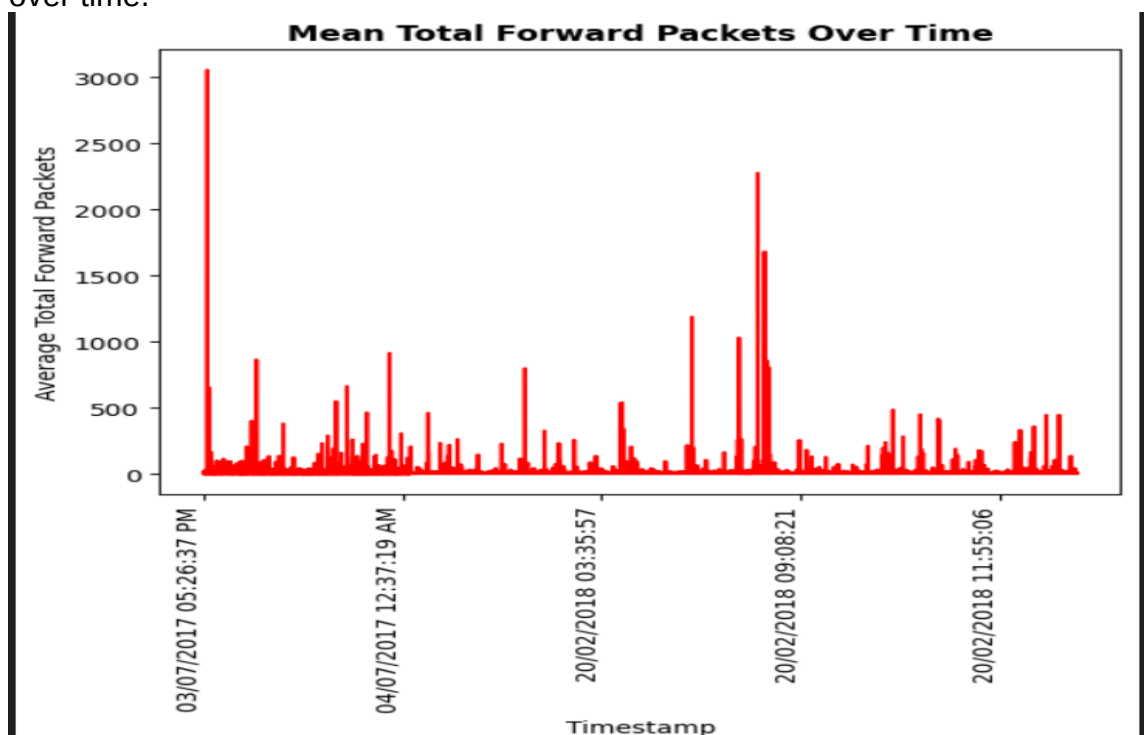


Figure 16 : Time series analysis

- A heatmap was made to find the correlations among different columns meaning how much a column's data would affect other column's data. This is highly efficient in order to find the inter-related values of a packet.

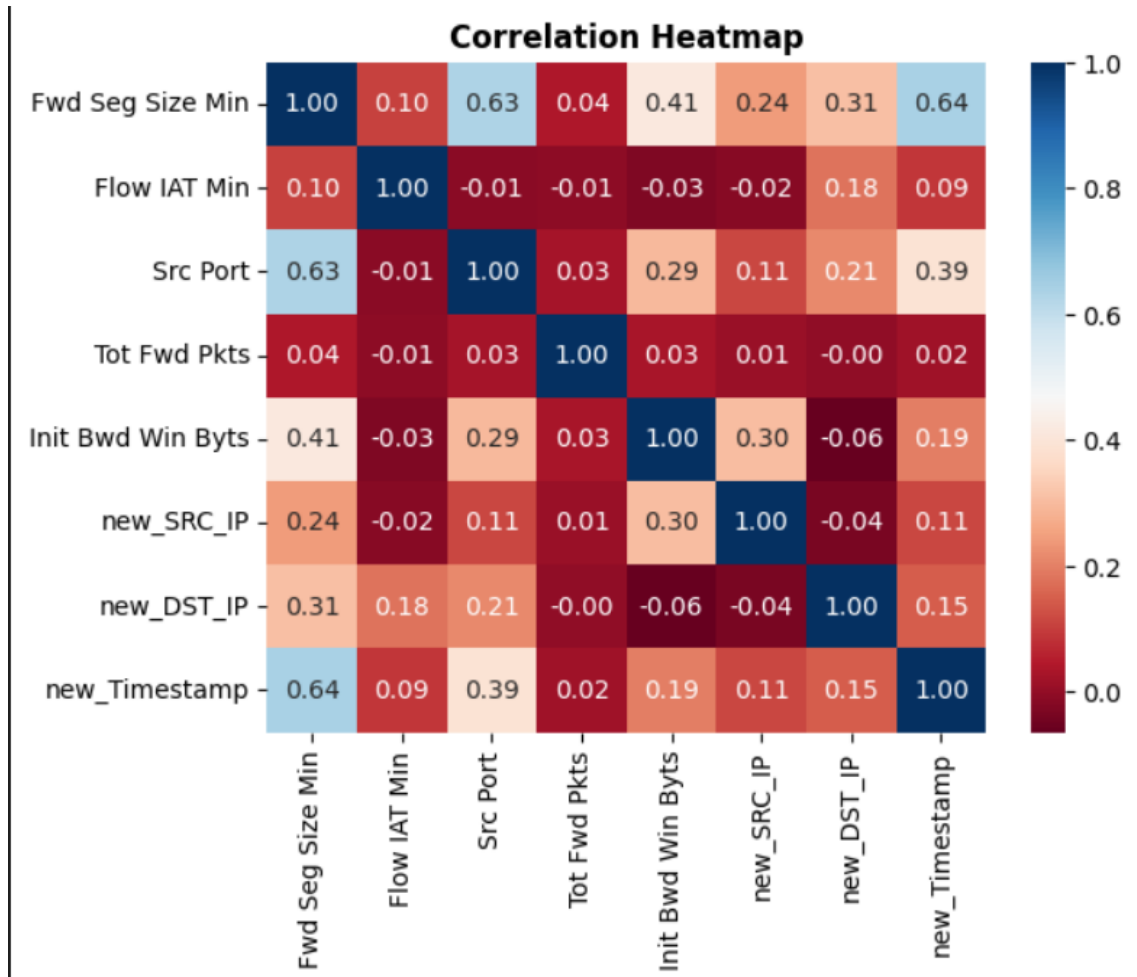


Figure 17 : Heatmap

All these graphs and maps explain the performance of the model and the patterns recognised by the models for detection.

Chapter 4: Discussion

After observation, it was seen that the final multi layered model was 90% accurate. Along with the final model, other models that were used also showed similar percent of accuracy. This indicated that all the models worked precisely similar. Yet, there were some aspects of the model that needs to be worked upon to enhance the detection and efficiency to ensure that the model would survive in a real-life network situation.

This model is a proof that the concept of machine learning could be highly beneficial for cyber security, by reducing most of the manual work and constantly learning nature of these models would combat the major threat of DDoS attacks.

4.1 Critical analysis

Critical analysis aims to understand the strengths and weaknesses of the project. Understanding these limitations and focusing on eliminating them could make the model a perfect fit for real life application. This is a key step to dive deeper into model's performance and analyse its limitations as the model is critically analysed in this section.

4.1.1 Weakness and limitations of this model

This model is highly accurate, yet there could be times when the model misclassifies a packet as it is 90% accurate. This could result in blockage of legit users. Hence more fine tuning of hyperparameters needs to be done in order to make it efficient in real world situation.

A major limitation would be that the machine learning system is trained on already collected dataset meaning it would need to adapt to the real-world data collection and adapt to certain new variables that would be introduced if the network is live. But a major strength of this model would be that it constantly learns the new pattern of new datasets making it highly adaptable in new situations, yet it could take some time for the system to manage live networks.

Another limitation would be that the source IP, destination IP and timestamps were not efficiently converted to integer values making the accuracy of model fall. Hence more efficient and concise coding could result in a better working model and data preprocessing must be done logically. Improving data preprocessing step could directly impact model's performance hence should be done cautiously.

4.1.2 Strengths of this model

Despite all these limitations, this machine learning model has many advantages over the traditional methods.

- These models are well optimized to prevent misclassifications, with being 90% accurate, and have a room for further development increasing its potential.
- Apart from this, there are several models that are involved in this project having their own advantages. Hence, making the overall system fast classifier, effective in anomaly detection, computationally efficient and works well with large amount of data.
- This model is proven to be accurate on imbalanced dataset, as the testing set that was used for model testing was highly imbalanced yet, the model performed well and predicted values maintaining a high accuracy.
- The model has the ability to adapt to new threat patterns while handling large datasets and live network packets. This ensures that the model is highly scalable and effective making them of high value for the modern real life security applications.

These are some highlights of this project proving that involvement of machine learning in security field is very beneficial given the traditional ways of attack detection. There are many more statistical proofs claiming that these modern solutions are of great importance for preventing the system from modern attacks.

More advancements in machine learning field could result in eradicating all these limitations making the machine learning models a perfect solution. But still, there are many limitations that can be prevented if the model is approached logically and properly with high cautious. This can be done once the model is produced. Doing further work on these models could be beneficial in deepening understanding of working of these models.

Chapter 5: Further work

While this model is performing well and producing results maintain a high accuracy, there are some fields which requires some further work in order to reach its model's full potential. There is potential work varying from improving the model's performance to application of model in outside environment.

- Addressing the misclassifications by the model could be prevented by performing a deep clean on dataset. Data preprocessing and feature selection could be done more logically to enhance model's performance. Or using even more datasets could result in exposure of model into many types of patterns in which attack happens.
- Tuning the hyperparameters could also result in enhanced performance by the model. This could be done after all the other work is done to push the model to its full potential
- Using the 'pickle' module, this model could be saved on the system. Meaning deployment of the model could take place. For that, understanding about the pickle module is necessary. Hence, this could be the further research that would be helpful in making the DDoS detection model go live on network.
- This model could be integrated into a firewall. This firewall would act as an active safeguard to secure the network by scanning the network and dropping any packet that will be classified as malicious. This could result in a live solution that can be implemented in real network environment. This will be of high usage as this could prevent any future DDoS attacks on the network and can alert the system about the source of attack benefitting the network, organizations and individuals.

Chapter 6: Conclusion

This project successfully developed a classification model to accurately predict the DDoS and Benign packets. This project demonstrates usage of machine learning in preventing a growing threat of DDoS attacks which disrupts the network. Exploration of many machine learning models was done to select and combine for the final model. The final model was a multi layered model using the results from previous models making the predictions highly accurate. This project proves that machine learning is an efficient and scalable approach for classification of DDoS packets and Benign packets. Comparing the performance and results with the traditional techniques, it was observed that this model had many advantages with slight limitations. These limitations could be prevented and fixed with some further work, resulting in enhanced detection, adaptability and accuracy of the model. However, several challenges had to be overcome in order to produce the final working model. Machine learning's constant learning nature makes it highly adaptable resulting in combatting the evolving threat with unknown patterns. With some further work, this model would be an improved and improvised version which will be ready to deploy in the real time network scenario. This work would require some more research on this topic but would result in solving one of the biggest threats to cybersecurity. Hence, this project successfully proves the significance of using machine learning models in cybersecurity scene.

References

- [1] Wang, W. and Lu, Z. (2018) 'Cyber security in the smart grid: Survey and challenges', *Computer Networks*, 57(5), pp. 1344–1371.
- [2] Sharafaldin, I., Habibi Lashkari, A. & Ghorbani, A.A. (2018) 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pp. 108–116.
- [3] Mirkovic, J. & Reiher, P. (2004) 'A taxonomy of DDoS attack and DDoS defense mechanisms', *ACM SIGCOMM Computer Communication Review*, 34(2), pp. 39-53.
- [4] Santos, Reneilson, et al (2019). "Machine Learning Algorithms to Detect DDoS Attacks in SDN." *Concurrency and Computation Practice and Experience*.
- [5] Buczak, A.L. and Guven, E. (2016) 'A survey of data mining and machine learning methods for cyber security intrusion detection', *IEEE Communications Surveys & Tutorials*, 18(2), pp. 1153–1176.
- [6] Somani, Gaurav, et al (2017). "Combating DDoS Attacks in the Cloud: Requirements, Trends, and Future Directions." *IEEE Cloud Computing*.
- [7] Singh, Anshuman, and Brij B. Gupta (2022). "Distributed Denial-of-Service (DDoS) Attacks and Defense Mechanisms in Various Web-Enabled Computing Platforms." *International Journal on Semantic Web and Information Systems*.
- [8] Doriguzzi-Corin, Roberto, et al (2020). "Lucid: a Practical, Lightweight Deep Learning Solution for DDoS Attack Detection." *IEEE Transactions on Network and Service Management*.
- [9] Behal, S. and Kumar, K. (2017) 'Trends in validation of DDoS research', *Procedia Computer Science*, 112, pp. 1283–1292.
- [10] Parashar, Anubha, et al (2023). "Data Preprocessing and Feature Selection Techniques in Gait Recognition: a Comparative Study of Machine Learning and Deep Learning Approaches." *Pattern Recognition Letters*.
- [11] Mohammed, Ameer Sameer Hamood (2024). "Detection and Prevention of Distributed Denial of Service (DDoS) Attacks Using Metaheuristic and Machine Learning Techniques." *International Journal of Science and Research (IJSR)*.
- [12] Zaidi, Syed Ali Nawaz (2025). "Comprehensive Approaches for Detecting and Mitigating Distributed Denial of Service (DDoS) Attacks in Modern Network Environments." *International Journal for Research in Applied Science and Engineering Technology*.
- [13] Yan, Q. and Yu, F. (2015). 'Distributed denial of service attacks in software-defined networking with cloud computing', *IEEE Communications Magazine*.